

**UNIVERSIDAD DE SANTIAGO DE CHILE  
DEPARTAMENTO DE INGENIERÍA MECÁNICA  
INGENIERÍA CIVIL MECATRÓNICA**



## **Estructura de Computadores y Sistemas Distribuidos**

### **Evaluación 3 — Caso 2: Línea de Producción con Balanceo Dinámico**

**Sistema distribuido de simulación de una línea conservera con coordinación por colas,  
detección de cuellos de botella en vivo y tablero de operaciones**

#### **Integrantes:**

César Olivares  
Simón García-Huidobro

#### **Profesor:**

Daniel Calderón R.

**Santiago, Chile  
23 de agosto de 2026**



## Índice

|                                                                                |           |
|--------------------------------------------------------------------------------|-----------|
| <b>1. Introducción</b>                                                         | <b>2</b>  |
| <b>2. Arquitectura del sistema</b>                                             | <b>3</b>  |
| 2.1. Servicios . . . . .                                                       | 3         |
| 2.2. Nodos lógicos y su justificación . . . . .                                | 3         |
| 2.3. Flujo de una orden . . . . .                                              | 4         |
| <b>3. Decisiones de diseño</b>                                                 | <b>4</b>  |
| 3.1. Balanceo por demanda: cola <i>pull</i> , no balanceador clásico . . . . . | 4         |
| 3.2. Coordinación y condición de carrera: provocarla antes de resolverla . . . | 5         |
| 3.3. Criterio de cuello de botella: espera, no servicio . . . . .              | 6         |
| 3.4. Persistencia: dos tablas y nada guardado dos veces . . . . .              | 7         |
| 3.5. Interfaz de operación . . . . .                                           | 7         |
| <b>4. Resultados experimentales</b>                                            | <b>7</b>  |
| 4.1. Experimentos 1–3: el cuello se desplaza al escalar . . . . .              | 8         |
| 4.2. Experimento 4: balanceo dinámico con una réplica lenta . . . . .          | 9         |
| 4.3. Experimento 5: saturación y recuperación automática . . . . .             | 10        |
| <b>5. Manejo de errores</b>                                                    | <b>11</b> |
| <b>6. Reproducibilidad y despliegue</b>                                        | <b>11</b> |
| <b>7. Resiliencia: cómo se abordaría</b>                                       | <b>12</b> |
| <b>8. Análisis crítico de dificultades</b>                                     | <b>12</b> |
| <b>9. Supuestos declarados</b>                                                 | <b>13</b> |
| <b>10. Conclusiones</b>                                                        | <b>14</b> |

## 1. Introducción

Este informe presenta el diseño, implementación y evaluación experimental de un sistema distribuido que simula una **línea de producción con balanceo dinámico de carga** (Caso 2). El dominio elegido es una planta conservera **ficticia** de jurel en lata de 425 g, inspirada en la industria conservera nacional, con cuatro etapas en serie separadas por **colas de trabajo**: cada cola es, literalmente, la fila de lotes que esperan frente a la etapa siguiente. Si una etapa procesa más lento de lo que le llega trabajo, su fila crece — y esa fila creciente es exactamente lo que este sistema mide y diagnostica en vivo.

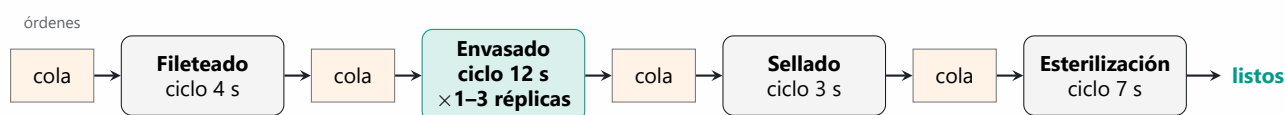


Figura 1: La línea de producción: cuatro etapas en serie unidas por colas de trabajo. Envasado, la etapa más lenta, es la replicada.

Tabla 1: Etapas de la línea y tiempos de ciclo simulados (supuesto declarado, §9).

| Etapa           | Naturaleza            | Ciclo (s/lote) | Réplicas                         |
|-----------------|-----------------------|----------------|----------------------------------|
| Fileteado       | Manual, paralelizable | 4              | 1                                |
| <b>Envasado</b> | Máquina continua      | <b>12</b>      | <b>1-3</b><br><b>(replicada)</b> |
| Sellado         | Máquina rápida        | 3              | 1                                |
| Esterilización  | Autoclave, ciclo fijo | 7              | 1                                |

La unidad de trabajo es el **lote** (una cantidad de latas del mismo formato). El sistema cumple los requisitos del enunciado: arquitectura distribuida con más de tres servicios en tres nodos lógicos, una etapa replicada detrás de un mecanismo de balanceo, coordinación del reparto sin condiciones de carrera, detección de cuellos de botella *en vivo*, persistencia, interfaz de operación con interacción real, y despliegue completo con un solo comando (`docker compose up`).

El resultado experimental central (§4) demuestra literalmente la premisa del caso: al escalar envasado de 1 a 2 réplicas, el cuello de botella **se desplaza** de envasado a esterilización y el lead time cae un 21 %; una tercera réplica ya casi no mejora nada, porque el límite se mudó de etapa.

## 2. Arquitectura del sistema

### 2.1. Servicios

Tabla 2: Servicios del sistema. Todos en Python; API con FastAPI, UI con Streamlit.

| Servicio        | Responsabilidad                                                                              | Tecnología               |
|-----------------|----------------------------------------------------------------------------------------------|--------------------------|
| orders-api      | Crea órdenes, las persiste y las encola                                                      | FastAPI + SQLite + Redis |
| estacion (×4–6) | Procesa lotes; una imagen única configurada por variables de entorno                         | Python + Redis + SQLite  |
| metrics-api     | Calcula espera, servicio y lead time; detecta el cuello de botella                           | FastAPI + SQLite + Redis |
| ui              | Tablero de operaciones: timeline, carga por réplica, alertas, histórico, creación de órdenes | Streamlit                |
| Redis           | Colas de trabajo y estado de réplicas                                                        | Redis 7                  |

Las cuatro estaciones son **un solo programa** desplegado cuatro (o más) veces: la etapa, sus colas de entrada/salida y su tiempo de ciclo llegan por variables de entorno (NOMBRE\_ETAPA, COLA\_ENTRADA, COLA\_SALIDA, TIEMPO\_CICLO). Escalar la etapa replicada es `docker compose up --scale envasado=3`: ninguna réplica sabe cuántas hermanas tiene ni necesita saberlo.

### 2.2. Nodos lógicos y su justificación

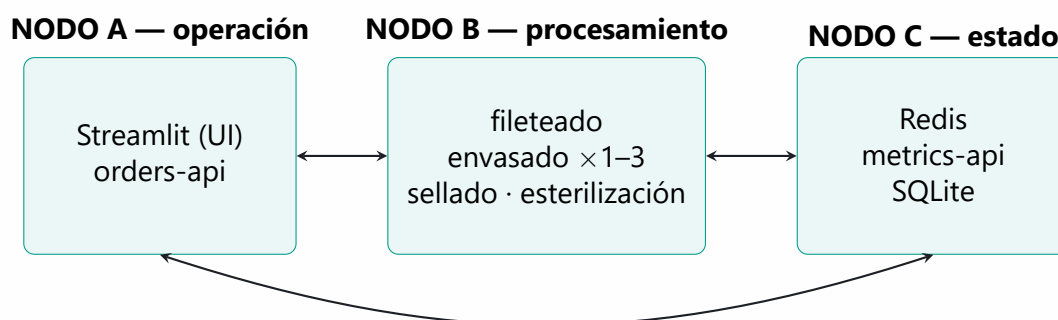


Figura 2: Los tres nodos lógicos y sus comunicaciones (HTTP/JSON y protocolo Redis).

La agrupación no es arbitraria: cada nodo reúne responsabilidades que **cambian de escala por motivos distintos**, el criterio clásico para particionar un sistema distribuido [1]. El nodo B es el único que se replica bajo carga (agregar capacidad de procesamiento



= agregar contenedores ahí). El nodo C concentra el estado compartido, que por ser compartido *no puede* duplicarse sin coordinación adicional. El nodo A expone el sistema al operador y escala con los usuarios, no con la producción.

**Por qué mantener dos o más nodos** (exigencia de la rúbrica): además de ser requisito del enunciado, separa dominios de falla y de recursos — la saturación de CPU de las estaciones (nodo B) no degrada la atención HTTP del operador (nodo A) ni la integridad del estado (nodo C); y permite dimensionar hardware distinto para cargas distintas (B necesita CPU, C necesita disco confiable). La sección §7 discute cómo se abordaría la resiliencia sobre esta misma división.

### 2.3. Flujo de una orden

Una orden nace en `orders-api` (`POST /ordenes`): se inserta en SQLite, se registra su evento `entraCola` y su `id` se empuja a `cola:fileteado` — todo dentro de una transacción, de modo que si Redis no responde no quedan “órdenes fantasma” guardadas pero nunca encoladas. Cada estación repite el mismo ciclo: reclama un `id` de su cola de entrada (`BRPOP`), registra `inicia`, simula el ciclo (`sleep(TIEMPO_CICLO)`), registra `termina` y `entraCola` de la etapa siguiente, y empuja el `id` a la cola siguiente. La última etapa marca la orden como `terminada`.

## 3. Decisiones de diseño

### 3.1. Balanceo por demanda: cola *pull*, no balanceador clásico

El enunciado sugiere “réplicas detrás de un balanceador”. Decidimos —y es una **decisión nuestra que defendemos aquí**— implementar el balanceo como **cola de trabajo compartida**: las réplicas *piden* un lote cuando quedan libres, en vez de recibirlo asignado por una pieza intermedia — el patrón de consumidores en competencia sobre un broker de mensajes [2].

1. Es lo único que produce balanceo **dinámico** real. Un round-robin le seguiría enviando lotes a una réplica lenta o atascada; con reparto por demanda, la réplica lenta simplemente pide menos veces. El experimento 4 (§4.2) lo muestra con datos: 11/12/7.
2. Resuelve la condición de carrera **con el mismo mecanismo**: la extracción atómica de la cola es a la vez el reparto y la garantía de exclusión (§3.2), sin una pieza extra que configurar ni un lock con expiración que se pueda vencer en el momento

equivocado.

3. El reparto emergente es proporcional a la capacidad real de cada réplica, sin conocer cuántas hay: `--scale envasado=3` funciona sin tocar configuración.

El “balanceador” del enunciado existe, pero es la cola: el punto único por el que pasa el trabajo y que decide quién procesa qué. Si se exigiera un balanceador HTTP clásico, un Nginx delante de las réplicas cubriría las consultas de estado; para el *trabajo*, el reparto por demanda es estrictamente superior en este caso.

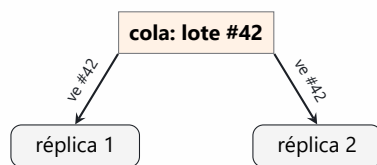
### 3.2. Coordinación y condición de carrera: provocarla antes de resolverla

El requisito de coordinación (“que dos réplicas no tomen la misma orden”) se abordó en dos pasos deliberados, ambos preservados en el historial de commits:

**Versión ingenua (commiteada rota a propósito).** Reclamar una orden en dos operaciones: mirar la cola (LRANGE) y después sacar el elemento (LREM). Entre ambas hay una ventana en la que otra réplica ve *la misma* orden. Con 3 réplicas y 200 órdenes, el experimento reprodujo órdenes procesadas dos veces — la evidencia de que el problema es real y no teórico.

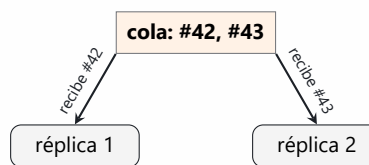
**Versión atómica.** BRPOP: extracción bloqueante en **una sola operación indivisible** [5]. Redis, al ser monohilo en la atención de comandos, garantiza que cada elemento se entrega a exactamente una réplica aunque haya varias esperando sobre la misma cola. Mismo experimento: **0 duplicados en 200 órdenes.**

Versión ingenua: mirar y sacar (2 pasos)



ambas procesan #42: duplicado

Versión atómica: BRPOP (1 paso)



cada lote a exactamente una réplica

Figura 3: La condición de carrera y su solución. A la izquierda, entre “mirar” y “sacar” hay una ventana en la que dos réplicas ven el mismo lote; a la derecha, la extracción es una única operación indivisible y la ventana no existe.

Se prefirió la operación atómica de cola por sobre un lock/contador distribuido (la alternativa que el enunciado menciona como ejemplo) porque elimina la ventana en vez de administrarla: no hay TTL que expire a mitad de un ciclo ni liberación de lock

que se pueda olvidar. El requisito del “lock/contador distribuido” está cubierto por el mismo Redis que serializa las extracciones.

### 3.3. Criterio de cuello de botella: espera, no servicio

El enunciado no define qué es un cuello de botella; el criterio es una **decisión nuestra** y es el argumento técnico central del proyecto:

El cuello de botella es la etapa con **mayor tiempo de espera promedio en una ventana móvil**, siempre que supere su umbral de advertencia. Si ninguna lo supera, no hay cuello (cuello: null).

Con los eventos de cada orden se definen, por etapa:

$$\text{espera} = t_{\text{inicia}} - t_{\text{entra\_cola}}, \quad \text{servicio} = t_{\text{termina}} - t_{\text{inicia}}, \quad \text{lead time} = t_{\text{fin}} - t_{\text{creada}}$$

Un tiempo de *servicio* alto solo dice que la tarea es larga; el atasco es trabajo llegando más rápido de lo que la etapa lo drena, y eso se manifiesta en la *espera* — la misma relación entre llegadas, capacidad y fila que formaliza la ley de Little [4] y que la teoría de las restricciones usa para localizar la limitante de un proceso [3]. La analogía cotidiana: en un supermercado, que un cajero sea lento no significa que haya atasco; el atasco es la *fila* que crece frente a su caja. El experimento 2 lo ilustra con datos (Figura 4): la etapa señalada como cuello (esterilización, ciclo 7 s) tiene un ciclo **más corto** que envasado (12 s) — un criterio por tiempo de servicio jamás la habría encontrado.

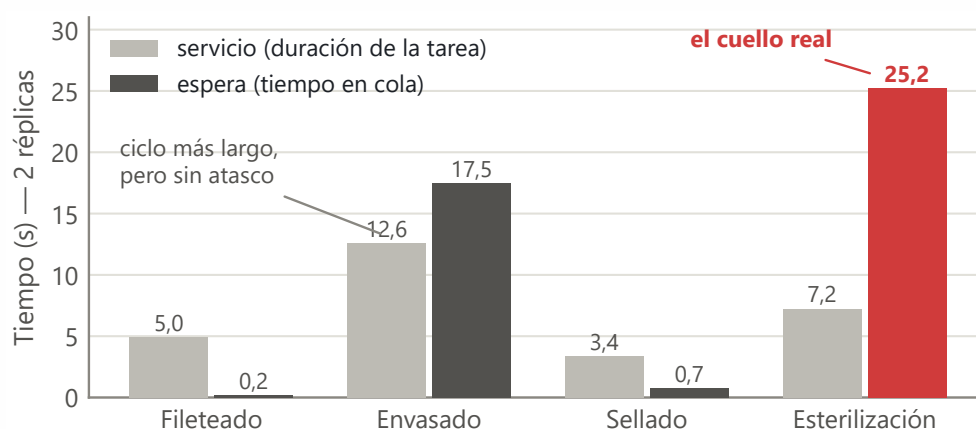


Figura 4: Servicio contra espera por etapa con 2 réplicas de envasado (experimento 2). Envasado tiene el ciclo más largo, pero la fila que crece — el cuello real — está en esterilización. Por eso el detector mide espera.

SQL: `SELECT * FROM DUAL WHERE 1=1`

[illegible]



cuya capacidad queda *bajo* la línea de llegada acumula fila tarde o temprano — y con 2 réplicas envasado sube a 10 lotes/min: mejor, pero aún insuficiente, lo que anticipa los resultados. Datos crudos en experimentos/resultados/.

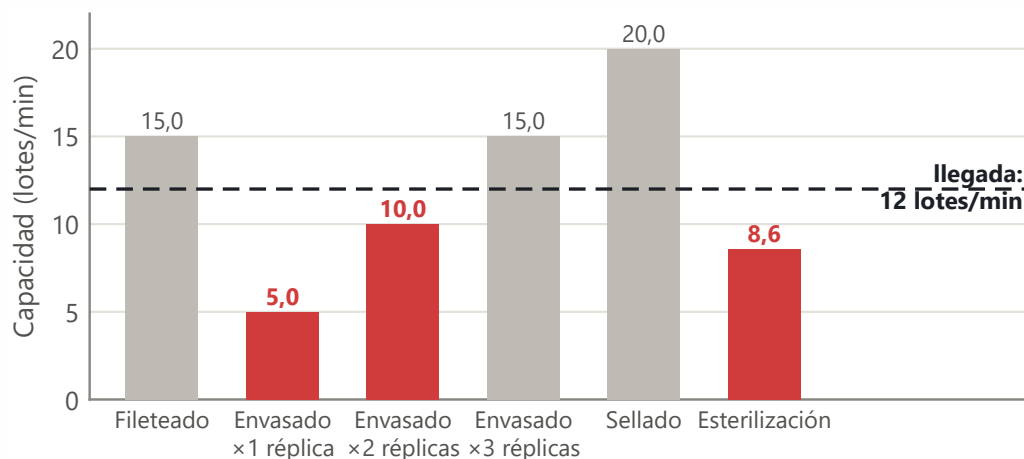


Figura 5: Capacidad de cada etapa (lotes/min) frente al ritmo de llegada del protocolo. Las etapas bajo la línea segmentada no alcanzan a drenar lo que les llega: ahí se forman las filas.

#### 4.1. Experimentos 1–3: el cuello se desplaza al escalar

Tabla 3: Resultado central: espera promedio por etapa (s) según réplicas de envasado.

| Réplicas | Fileteado | Envasado               | Sellado | Esterilización         | Lead time |
|----------|-----------|------------------------|---------|------------------------|-----------|
| 1        | 0,66      | <b>67,78 (crítico)</b> | 0,68    | 0,88                   | 82,9 s    |
| 2        | 0,18      | 17,52                  | 0,74    | <b>25,21 (crítico)</b> | 65,6 s    |
| 3        | 0,22      | 0,73                   | 0,89    | <b>34,50 (crítico)</b> | 62,0 s    |

- **1 réplica:** envasado acumula 14 lotes en cola y espera de 67,8 s; el resto de la línea, ociosa a ratos, espera < 1 s.
- **2 réplicas:** el reparto por demanda divide el trabajo (13/12 lotes por réplica) y el tiempo efectivo de envasado baja a ~6 s por lote. El cuello **se desplaza a esterilización** — la premisa del caso, demostrada con datos. Lead time: –21 %.
- **3 réplicas:** envasado queda con capacidad sobrada (espera 0,7 s), pero el lead time solo mejora otro 5 %: el límite ya no está ahí. La inversión correcta siguiente sería una segunda autoclave, no una tercera envasadora.

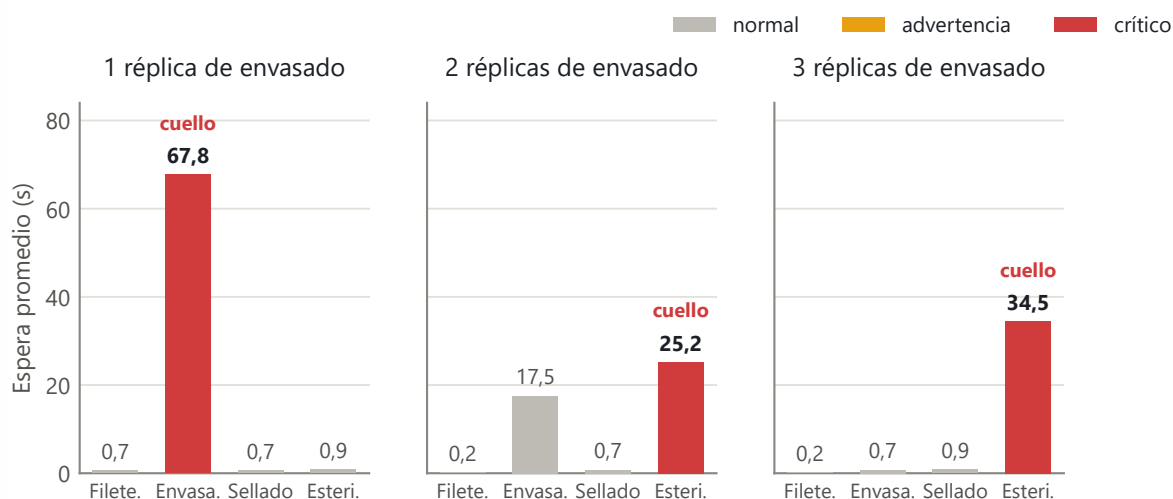


Figura 6: El resultado central, visto de una vez: la espera promedio por etapa según el número de réplicas de envasado. El cuello (barra roja) no desaparece al escalar: se desplaza de envasado a esterilización.

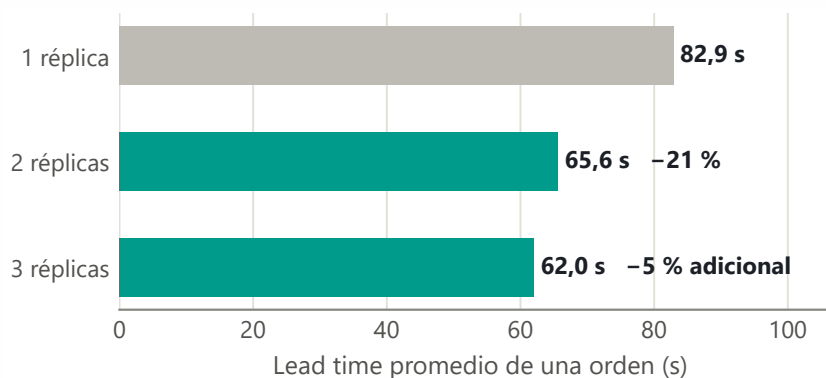


Figura 7: Lead time promedio de una orden (creación → terminada). La segunda réplica compra un 21 %; la tercera casi nada, porque el límite ya no está en envasado.

Los tiempos de servicio medidos (4,6–5,0 / 12,2–13,0 / 3,3–3,5 / 7,3 s) coinciden con los ciclos configurados (4/12/3/7 s), lo que valida la instrumentación.

#### 4.2. Experimento 4: balanceo dinámico con una réplica lenta

Dos réplicas normales más una configurada al doble de ciclo, sobre la misma cola, 30 lotes: la lenta procesó **7** contra **11 y 12** de las normales (Figura 8). Un round-robin habría repartido 10/10/10 sin importar la velocidad. Nadie asignó ese reparto: emergió de que cada réplica reclama trabajo solo al quedar libre.

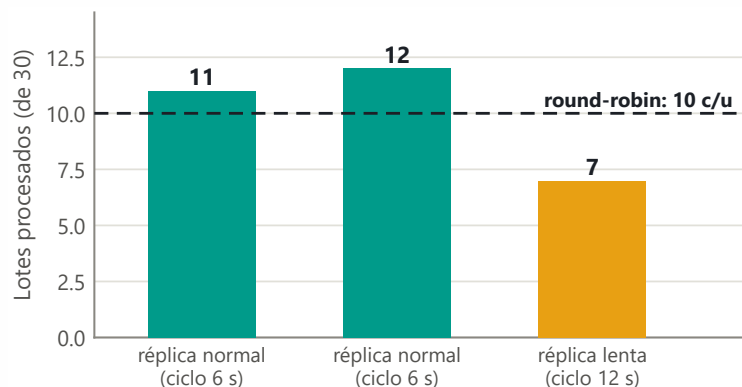


Figura 8: Balanceo dinámico con una réplica al doble de ciclo: el reparto se adapta solo a la capacidad real de cada una, sin configuración ni coordinación explícita.

### 4.3. Experimento 5: saturación y recuperación automática

Ráfaga de 25 órdenes de golpe (envasado  $\times 2$ ), observando `/metricas/cuello` cada 15 s: a los 17 s la primera etapa pasa a advertencia y a los 32 s a crítico — **sola**. La Figura 9 muestra la película completa: la congestión recorre la línea como una ola (fileteado  $\rightarrow$  envasado  $\rightarrow$  esterilización) y a los **278 s** las cuatro etapas están de vuelta en normal **sin intervención alguna**: las esperas viejas salieron de la ventana móvil. Detalle en `experimentos/resultados/exp5-saturacion.txt`.

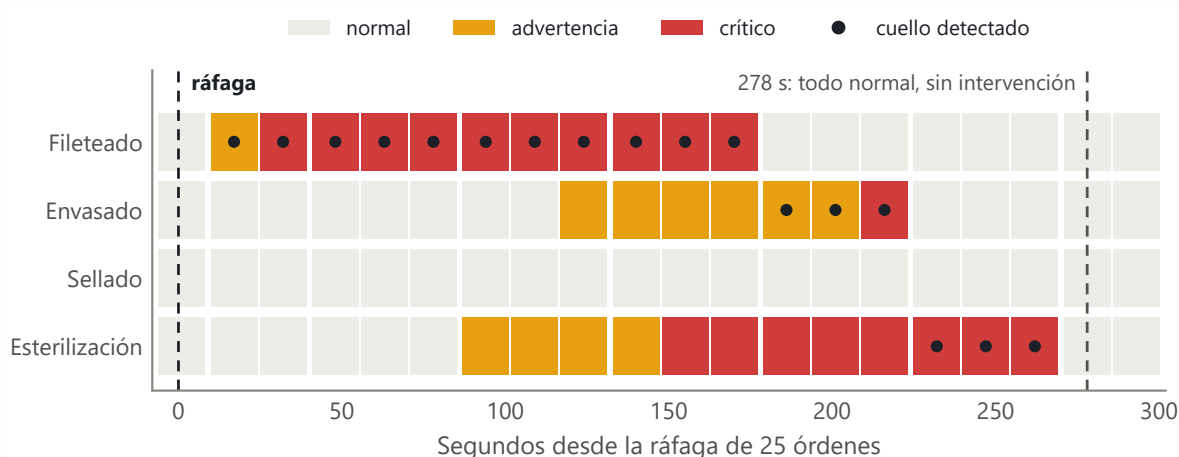


Figura 9: Experimento 5 como línea de tiempo de estados: cada fila es una etapa y cada celda una sonda de `/metricas/cuello`. La congestión avanza por la línea y el sistema vuelve a normal por sí solo.

La ráfaga deja además una lección de operación: con todo inyectado a la vez, el cuello detectado inicialmente es la *primera* etapa (ahí se apila la cola), no la de mayor ciclo.

El cuello depende del patrón de llegada, no solo de los tiempos de proceso — por eso el protocolo de los experimentos 1–3 inyecta a ritmo constante.

## 5. Manejo de errores

Códigos verificados contra el sistema corriendo: 201 (creación), 422 (cantidad  $\leq 0$ , producto vacío, filtro inválido — nunca 500), 404 (orden inexistente), 503 (dependencia caída, con mensaje en castellano). La UI nunca muestra un traceback: ante cualquier falla consulta la salud de los tres servicios y **nombra al que realmente falta** — si Redis cae, los demás siguen vivos y culpar al que responde lento enviaría a buscar el error al lugar equivocado.

El hallazgo técnico de esta parte: con Redis detenido, los timeouts de socket del cliente **no bastan**, porque el cuelgue ocurre antes, en la resolución DNS (Docker elimina el nombre del contenedor detenido y `getaddrinfo` tarda  $\sim 45$  s en rendirse — medido). La solución fue un **plazo duro**: la consulta a Redis corre en un hilo aparte y se espera su resultado a lo más 2 s; si no llega, se responde 503 de inmediato. Resultado medido: `POST /ordenes` con Redis caído pasó de colgarse 46 s a responder 503 en **2,1 s**, y `/metricas` responde 200 en 2,0 s con `en_cola`: null (un null honesto; un 0 sería mentira).

Todos los servicios exponen `GET /salud` y tienen `healthcheck` en el compose. El `healthcheck` mide *vida*, no dependencias: un servicio que responde 503 explicándose está sano — marcar `unhealthy` servicios vivos habría provocado reinicios en cascada de contenedores que no tienen nada malo.

## 6. Reproducibilidad y despliegue

```
git clone <repositorio> && cd <carpeta>
docker compose up -d --build
# tablero en http://localhost:8501
```

Sin pasos manuales, sin instalar nada más que Docker [7]. La prueba se ejecutó de verdad: clon en carpeta vacía → 8/8 contenedores `healthy` → orden creada por API recorre las cuatro etapas hasta `terminada` → tablero arriba. Toda la configuración tiene defaults funcionales (`.env.example` documenta cada variable; no es obligatorio crearlo). El README cubre instalación, uso, configuración, experimentos y diagnóstico de fallas comunes.



## 7. Resiliencia: cómo se abordaría

El enunciado no exige tolerancia a fallos y este proyecto, deliberadamente, **detecta y explica** las fallas en vez de enmascararlas (sin reintentos, sin circuit breaker). Si se necesitara operar esto en producción, la arquitectura ya deja los puntos de corte correctos:

- **Estaciones (nodo B):** ya son resilientes en lo esencial — sin estado propio, reconectan solas (bucle de reintento del worker) y pueden multiplicarse o morir sin corromper nada. Lo único pendiente sería re-encolar el lote en curso si una réplica muere a mitad de ciclo (hoy ese lote se pierde de la cola; el patrón estándar es BLMOVE a una cola “en proceso” por réplica, devuelta a la cola principal por un barrendero al detectar un latido vencido).
- **Redis (nodo C):** es el punto único de falla del reparto. El camino estándar es Redis Sentinel (réplica + failover automático) o un broker con confirmaciones (el bonus Kafka apuntaba ahí): con *acks*, un mensaje no confirmado vuelve a entregarse solo.
- **SQLite (nodo C):** suficiente para un nodo; con más escritores o failover se migraría a MySQL/PostgreSQL gestionado, cambiando solo el módulo `bd.py` de cada servicio.
- **APIs y UI (nodo A):** sin estado; con dos instancias y un balanceador HTTP delante quedarían en alta disponibilidad. Los healthchecks ya existentes son el insumo de ese balanceador.

La división en nodos de §2.2 es exactamente la que hace estos upgrades locales: cada mejora toca un nodo sin rediseñar los demás.

## 8. Análisis crítico de dificultades

1. **El entorno costó más que el primer código.** La virtualización venía deshabilitada en BIOS (SVM Mode); `wsl --install` no instaló ninguna distribución (hubo que pedir Ubuntu aparte); y Docker Desktop caía en cascada por sockets huérfanos de un arranque fallido que Windows no podía borrar, sembrando cada caída el bloqueo de la siguiente. Lección: el “hola mundo” de infraestructura es parte del proyecto y hay que presupuestarlo.
2. **La condición de carrera hubo que fabricarla para verla.** Con el reclamo ingenuo, la ventana entre mirar y sacar es de microsegundos: sin duplicados no hay demostración. Se ensanchó con una espera artificial configurable



(DEMORA\_INGENUA) hasta reproducir duplicados de forma confiable, y ambas versiones quedaron commiteadas — entender el problema antes de resolverlo.

3. **“Le puse timeout” no es verificable hasta medir el camino completo de la falla.** El cuelgue con Redis caído no estaba en el socket sino en el DNS (46 s medidos), que el timeout configurado no cubría. Hubo que aislar la falla por capas (resolución → conexión → lectura) y acotar la operación completa con un plazo duro.
4. **Al acelerar una simulación se puede medir otra cosa.** El experimento de la réplica lenta con ciclos de  $\sim 1$  s dio un reparto casi parejo (21/20/19): a ese ritmo, el costo por lote lo dominaba la escritura de eventos en SQLite (seis procesos compitiendo por el lock sobre el volumen), no el ciclo simulado. Con ciclos de 6/12 s el overhead se vuelve despreciable y el resultado es el esperado (11/12/7). El overhead constante no escala con la simulación.
5. **El cuello de botella depende del patrón de llegada.** Inyectar lotes de golpe apila todo en la primera cola y el sistema señala —correctamente— a fileteado. Entender que eso no era un bug sino una propiedad de las colas cambió el protocolo experimental y el guion de la demo: ritmo constante para responder “¿qué etapa limita la producción?”.

## 9. Supuestos declarados

1. Los tiempos de ciclo (4/12/3/7 s) son inventados, elegidos para que el desplazamiento del cuello sea demostrable en una demo corta. No representan una planta real; el escenario es ficticio.
2. “Procesar un lote” se simula esperando el tiempo de ciclo; no hay trabajo real.
3. Un lote representa una cantidad de latas del mismo formato; no se modelan latas individuales.
4. El enunciado no fija qué persistir: decidimos órdenes y eventos, con las métricas siempre derivadas.
5. El enunciado no define el criterio de cuello de botella ni umbrales: el criterio de §3.3 es decisión nuestra.

## 10. Conclusiones

El sistema cumple los requisitos del caso de punta a punta y sus afirmaciones están respaldadas por mediciones reproducibles: el reparto por demanda balancea sin conocer las réplicas (11/12/7 con una réplica lenta), la extracción atómica elimina la condición de carrera (0 duplicados en 200 órdenes, contra duplicados reproducibles de la versión ingenua), el criterio de espera en ventana móvil detecta el cuello de botella correcto incluso cuando no es la etapa más lenta, y se recupera solo al bajar la carga. El resultado central — el cuello se desplaza de envasado a esterilización al escalar de 1 a 2 réplicas, y una tercera ya no compra casi nada — es la demostración empírica de que en una línea en serie *el cuello de botella no se elimina: se muda* [3], y de que medir espera (no servicio) es lo que permite verlo en vivo.

## Referencias

- [1] M. van Steen y A. S. Tanenbaum, *Distributed Systems*, 4.<sup>a</sup> ed. Maarten van Steen, 2023. Disponible en <https://www.distributed-systems.net>.
- [2] M. Kleppmann, *Designing Data-Intensive Applications*. Sebastopol, CA: O'Reilly Media, 2017.
- [3] E. M. Goldratt y J. Cox, *La Meta: un proceso de mejora continua*, 3.<sup>a</sup> ed. Great Barrington, MA: North River Press, 2004.
- [4] J. D. C. Little, "A proof for the queuing formula  $L = \lambda W$ ", *Operations Research*, vol. 9, n.º 3, pp. 383–387, 1961.
- [5] Redis Ltd., "BRPOP — Redis command reference", documentación oficial de Redis 7, 2024. Disponible en <https://redis.io/docs/latest/commands/brpop/>.
- [6] SQLite Consortium, "Write-Ahead Logging", documentación oficial de SQLite, 2024. Disponible en <https://www.sqlite.org/wal.html>.
- [7] Docker Inc., "Docker Compose overview", documentación oficial de Docker, 2024. Disponible en <https://docs.docker.com/compose/>.

## Anexos (seleccionar hasta 15 páginas)

- **A.** Contratos completos de las APIs (docs/disenio.md §1).
- **B.** Esquema SQL y momentos de escritura de eventos (docs/disenio.md §2).



- **C.** Capturas del tablero: línea normal, advertencia, crítico, Redis caído.
- **D.** Salidas crudas de los experimentos (experimentos/resultados/).
- **E.** Historial de commits (dos integrantes, repartidos en el tiempo).
- **F.** Bitácoras por fase (docs/fases/), incluida la reproducción de la condición de carrera.